

学业发展规划系统（PDP）技术说明文档

一、实验原理与内容

1.1 项目概述

学业发展规划系统（Personal Development Planning System，简称 PDP）是一个基于 Qt 6 C++ 框架的跨平台桌面应用程序。该系统面向高校学生，旨在提供一站式的学业发展管理平台，帮助学生系统化地管理课程学习、角色任职、成就荣誉、实践经历、课外活动、目标规划等多维度发展数据，并通过 AI 智能助手提供个性化的发展建议。

系统采用前后端一体化架构，在同一进程中同时运行 HTTP 服务器和桌面 GUI 客户端。后端基于 Qt 的 QHttpServer 模块提供 RESTful API 服务，前端既可通过内置的 Vue3 构建产物以网页形式访问，也可通过原生 Qt Widgets 桌面界面直接操作。这种设计使得系统既保留了原生桌面应用的交互体验，又具备了 Web 服务的可扩展性。

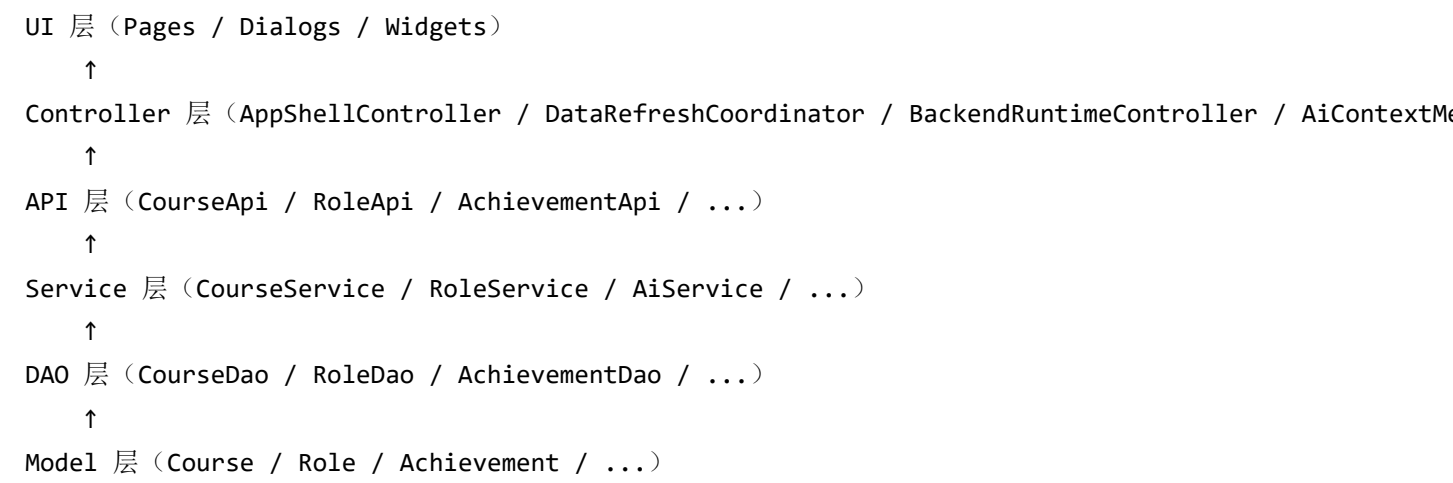
1.2 核心功能列表

系统包含以下核心功能模块：

- 课程管理**：记录课程名称、学分、学期、成绩、GPA 等信息，支持课程状态跟踪（计划中/进行中/已完成）和成绩绩点自动计算。
- 角色任职**：管理学生在组织中的各类角色，包括学生会、社团、班级干部等，记录任职时间、组织名称、职责描述。
- 成就荣誉**：记录学科竞赛获奖、专业证书获取、论文发表等成就，支持成果验证和级别分类。
- 实践经历**：管理实习、项目、科研等实践经历，记录技术栈、指导教师、成就产出等详细信息。
- 课外活动**：记录参与的课外活动，支持分类标记和收藏功能。
- 目标规划**：设定学业发展目标，支持量化指标追踪（目标值/当前值），提供进度可视化和里程碑管理。
- 岗位追踪**：记录目标岗位信息，包括公司、薪资范围、技能要求，支持需求达成度跟踪。
- 同学对照分析**：录入同学发展数据，进行 GPA、学分、成果数量等维度的横向对比分析。
- AI 智能助手**：集成大语言模型，支持学业分析、课程建议、目标建议、经历建议、简历建议等多种智能分析功能，同时提供本地规则模式作为离线回退方案。
- 简历生成**：基于系统中的课程、经历、成就等数据自动生成简历，支持 JSON 和 HTML 格式导出。

1.3 架构模式

系统采用**前后端一体化六层分层架构**，自底向上依次为：



每一层只依赖其直接下层，层间通过明确的接口进行通信。Model 层定义纯数据结构，DAO 层封装数据库访问，Service 层实现业务逻辑，API 层处理 HTTP 请求解析与响应构建，Controller 层协调 UI 与业务逻辑之间的流程，UI 层负责用户交互展示。

1.4 技术栈

类别	技术	版本
编程语言	C++	C++17
应用框架	Qt	6.5+（实测 6.11.0）
UI 框架	Qt Widgets	6.x
HTTP 服务器	QHttpServer	6.x
网络通信	QNetworkAccessManager	6.x
数据库	SQLite	3
数据库驱动	QSqlDatabase / QSqlQuery	6.x
构建工具	CMake	3.25+
AI 后端	Python + Qwen 2.5-1.5B	3.10+
前端构建	Vue3（预构建产物）	--

二、类清单与用途总览

2.1 自定义类

Model 层（12 个类）

类名	用途
Course	课程数据模型，包含课程名称、学分、成绩、GPA 等字段，提供 toDict() / fromDict() 序列化方法和 calculateGradePoint() 静态绩点计算方法
Role	角色任职数据模型，记录组织角色的标题、类型、组织名称、任职时间等
Achievement	成就荣誉数据模型，记录竞赛获奖、证书获取等成就信息，包含验证状态和级别分类
Experience	实践经历数据模型，记录实习、项目、科研经历的详细信息，包含技术栈和进行中状态
Activity	课外活动数据模型，记录活动名称、分类、标签等，支持收藏标记
Goal	目标规划数据模型，包含目标值/当前值的量化追踪，提供 progress() 方法计算完成百分比
Job	目标岗位数据模型，记录岗位标题、公司、薪资范围、技能要求等
JobRequirement	岗位技能需求数据模型，与 Job 表通过外键关联，记录技能名称和熟练度
PeerBenchmark	同学对照数据模型，记录同学的 GPA、学分、成果数量等用于横向对比
User	用户账户数据模型，包含用户名、邮箱、密码哈希、显示名称等
ImportResult	数据导入结果模型，汇总导入成功/失败数量
ImportErrorItem	数据导入错误条目模型，记录具体行号和错误信息

DAO 层（10 个类）

类名	用途
DaoBase	DAO 基类，持有 QSqlDatabase m_db 保护成员，构造时检查数据库连接状态
CourseDao	课程数据访问对象，提供 getAll() / getById() / create() / update() / remove() 等 CRUD 方法
RoleDao	角色任职数据访问对象

类名	用途
AchievementDao	成就荣誉数据访问对象
ExperienceDao	实践经历数据访问对象
ActivityDao	课外活动数据访问对象
GoalDao	目标规划数据访问对象
JobDao	目标岗位数据访问对象
PeerBenchmarkDao	同学对照数据访问对象
UserDao	用户账户数据访问对象，额外提供 <code>getByUsername()</code> / <code>getByEmail()</code> / <code>updateLastLogin()</code> 方法

Service 层（13 个类）

类名	用途
CourseService	课程业务逻辑，封装 CourseDao 调用并提供 GPA 统计
RoleService	角色任职业务逻辑
AchievementService	成就荣誉业务逻辑，提供成果统计和分类查询
ExperienceService	实践经历业务逻辑
ActivityService	课外活动业务逻辑
GoalService	目标规划业务逻辑
JobService	目标岗位业务逻辑
AIService	AI 智能助手核心服务，实现双模式架构（远程 AI 模型 / 本地规则回退）
AnalyticsService	数据分析服务，包含学期对比和同学对比分析（同时包含 PeerBenchmarkService）
AuthService	用户认证服务，实现密码哈希、Token 生成、注册登录逻辑
DashboardService	仪表盘服务，提供总览数据、GPA 趋势、推荐信息
ImportService	数据导入服务，支持 CSV 格式批量导入
ResumeService	简历生成服务，基于系统数据自动构建简历内容

API 层（14 个类）

类名	用途
CourseApi	课程 HTTP API 处理器，解析请求并调用 CourseService
RoleApi	角色任职 API 处理器
AchievementApi	成就荣誉 API 处理器
ExperienceApi	实践经历 API 处理器
ActivityApi	课外活动 API 处理器
GoalApi	目标规划 API 处理器
JobApi	目标岗位 API 处理器
DashboardApi	仪表盘 API 处理器
AnalyticsApi	数据分析 API 处理器
TimelineApi	时间线 API 处理器
ImportApi	数据导入 API 处理器
ResumeApi	简历生成 API 处理器
AiApi	AI 分析 API 处理器
AuthApi	用户认证 API 处理器

Server 层（1 个类）

类名	用途
HttpServer	HTTP 服务器封装类，组合 QHttpServer 和 QTcpServer，注册所有 REST 路由，提供静态文件服务

Controller 层（5 个类）

类名	用途
AppShellController	应用外壳控制器，管理页面切换时的顶栏标题更新和内容宽度调整
DataRefreshCoordinator	数据刷新协调器，基于 DataDomain 枚举驱动各页面的数据刷新

类名	用途
BackendRuntimeController	后端运行时控制器，管理 HttpServerThread 的生命周期，绑定状态标签和进度条
AiContextMediator	AI 上下文中介器，拦截全局文本选中事件并传递给 AI 面板
CrudPageController	CRUD 页面通用控制器，通过 std::function 回调注入对话框和服务实现

Page 层（13 个类）

类名	用途
BasePage	页面基类，定义 refresh() 纯虚函数和 createMetricCard() 工具方法
OverviewPage	总览页面，展示核心指标卡片和快捷操作
CoursesPage	课程管理页面
RolesPage	角色任职页面
AchievementsPage	成就荣誉页面
ExperiencesPage	实践经历页面
ActivitiesPage	课外活动页面
GoalsPage	目标规划页面
JobsPage	岗位追踪页面
AnalysisPage	数据分析页面
TimelinePage	时间线页面
ResumePage	简历生成页面
ImportsPage	数据导入页面

Dialog 层（9 个类）

类名	用途
ProfileEditorDialog	个人信息编辑对话框
CourseEditorDialog	课程编辑对话框

类名	用途
RoleEditorDialog	角色编辑对话框
AchievementEditorDialog	成就编辑对话框
ExperienceEditorDialog	经历编辑对话框
ActivityEditorDialog	活动编辑对话框
GoalEditorDialog	目标编辑对话框
JobEditorDialog	岗位编辑对话框
PeerEditorDialog	同学数据编辑对话框

Widget 层（17 个类）

类名	用途
SidebarWidget	侧边栏组件，包含导航列表和用户信息卡片
NavigationListWidget	导航列表组件，基于 QListWidget 实现页面切换
AiPanelWidget	AI 面板组件，支持展开/收起，包含快捷分析按钮和聊天输入
AiConversationWidget	AI 对话组件，展示 AI 分析结果
AiStatusBar	AI 状态栏组件，显示当前 AI 模式和连接状态
ToastNotification	消息提示组件，实现类似 Toast 的临时通知效果
TimeInfoCard	时间信息卡片，显示当前日期和学期信息
StudentInfoCard	学生信息卡片，展示用户基本信息
MetricGridWidget	指标网格组件，以卡片形式展示多项统计数据
SuggestionListWidget	建议列表组件，展示 AI 生成的个性化建议
CrudPageShell	CRUD 页面外壳组件，提供统一的表格布局和增删改查按钮
SemesterAnalysisWidget	学期分析组件，展示各学期 GPA 变化趋势
PeerBenchmarkWidget	同学对照组件，展示横向对比数据
ResumePreviewWidget	简历预览组件
ResumePreviewDialog	简历预览对话框

类名	用途
ResumeEditorPanel	简历编辑面板
ResumeCandidatePanel	简历候选数据面板

2.2 Qt 框架类

核心类

类名	使用场景
QMainWindow	MainWindow 继承自此类，提供菜单栏、工具栏、状态栏等主窗口框架
QWidget	所有页面和自定义组件的基类
QFrame	用于创建顶栏、卡片等带有边框的容器
QDialog	所有编辑对话框的基类
QObject	所有控制器类的基类，提供信号槽机制支持

布局类

类名	使用场景
QVBoxLayout	垂直布局，用于页面内容的纵向排列
QHBoxLayout	水平布局，用于侧边栏与内容区的横向排列、顶栏元素排列
QGridLayout	网格布局，用于指标卡片的多列排列
QFormLayout	表单布局，用于编辑对话框中标签-输入框的配对排列
QSplitter	分割器布局，用于可拖拽调整大小的区域分割

数据处理类

类名	使用场景
QSqlDatabase	数据库连接管理，通过 QSqlDatabase::database() 获取默认连接
QSqlQuery	SQL 查询执行，使用 prepare() + bindValue() 实现参数化查询
QJsonObject	JSON 对象表示，所有 Model 类的 toDict() / fromDict() 方法的核心数据结构
QJsonArray	JSON 数组表示，用于 tags、milestones 等多值字段的序列化

类名	使用场景
QJsonDocument	JSON 文档解析与构建，用于 HTTP 请求/响应的 JSON 序列化
QSettings	应用配置持久化，存储窗口几何状态等用户偏好

网络类

类名	使用场景
QHttpServer	HTTP 服务器核心类，注册路由并处理请求
QTcpServer	TCP 服务器，QHttpServer 的底层传输依赖
QNetworkAccessManager	HTTP 客户端，用于 AiService 向远程 AI 服务器发送请求

UI 组件类

类名	使用场景
QTableWidget	数据表格展示，用于课程列表、成就列表等结构化数据展示
QStackedWidget	页面堆栈管理，MainWindow 中用于切换不同功能页面
QSystemTrayIcon	系统托盘图标，支持最小化到托盘和托盘右键菜单
QProgressBar	进度条，用于显示后端服务器启动进度
QLabel	文本标签，用于顶栏标题、状态栏信息、指标卡片数值等
QToolBar	工具栏，提供刷新和网页预览快捷按钮
QScrollArea	滚动区域，用于页面内容超出可视区域时的滚动支持
QGraphicsOpacityEffect	透明度效果，配合 QPropertyAnimation 实现页面切换的淡入动画

2.3 C++ 标准库类

类名	使用场景
std::function	通用回调函数包装器。在 CrudPageController 中用于封装对话框创建和服务调用的函数签名（如 CreateDialogFn = std::function<QJsonObject(QWidget*)>）；在 MainWindow 的 m_pageRefreshers 中用于存储页面刷新回调的注册表（QMap<int, std::function<void()>>），替代 switch-case 硬编码

类名	使用场景
std::sort	通用排序算法。在数据分析页面中用于对课程按 GPA 降序排序、对目标按优先级排序等场景，配合自定义 lambda 比较器实现多字段排序
std::min	最小值计算。用于限制分页查询的返回数量、截断超长文本显示等场景
std::move	移动语义。在 JsonUtils::applyCommonHeaders() 中用于高效转移 QHttpHeaders 对象的所有权，避免不必要的拷贝

三、主要类的详细说明

3.1 核心数据模型类

Course 类

Course 是系统中最核心的数据模型类之一，定义在 src/model/Course.h 中。它以纯 C++ 类的形式（不继承 QObject）定义了课程的全部字段：

```
class Course {
public:
    int id = 0;
    QString name;           // 课程名称
    QString code;           // 课程代码
    double credits = 0;     // 学分
    QString semester;       // 学期（如 "2024-2025-1"）
    QString category = "Required"; // 分类（Required/Elective/Public）
    double score = 0;       // 成绩分数
    double gradePoint = 0;  // 绩点
    QString status = "Planned"; // 状态（Planned/In Progress/Completed）
    QString teacher;        // 授课教师
    QString location;        // 上课地点
    QString description;     // 课程描述
    QString tags;            // 标签（逗号分隔存储）
    QDateTime createdAt;
    QDateTime updatedAt;
};
```

序列化模式（toDict / fromDict）

每个 Model 类都遵循统一的序列化模式。toDict() 方法将对象转换为 QJsonObject，fromDict() 静态方法从 QJsonObject 还原对象。以 Course 为例：

```
QJsonObject toDict() const {
    QJsonObject obj;
    obj["id"] = id;
    obj["name"] = name;
    // ... 其他字段
    // tags 字段特殊处理：逗号分隔的字符串转为 JSON 数组
    QJsonArray tagsArr;
    for (const QString& t : tags.split(',', Qt::SkipEmptyParts)) {
        tagsArr.append(t.trimmed());
    }
    obj["tags"] = tagsArr;
    obj["createdAt"] = createdAt.toString(Qt::ISODate);
    obj["updatedAt"] = updatedAt.toString(Qt::ISODate);
    return obj;
}

static Course fromDict(const QJsonObject& obj) {
    Course c;
    c.id = obj["id"].toInt();
    // ... 其他字段
    // tags 字段反向处理：JSON 数组转为逗号分隔字符串
    if (obj["tags"].isArray()) {
        QStringList parts;
        for (const auto& v : obj["tags"].toArray()) {
            parts.append(v.toString());
        }
        c.tags = parts.join(",");
    } else {
        c.tags = obj["tags"].toString();
    }
    return c;
}
```

tags 和 milestones 等多值字段采用“数据库中以逗号分隔存储、JSON 序列化时转为数组”的策略，既简化了 SQLite 的存储结构，又为前端提供了结构化的 JSON 数组格式。

calculateGradePoint 静态方法

Course 类提供了一个静态方法用于根据分数计算绩点：

```
static double calculateGradePoint(double score, const QString& scale = "standard");
```

该方法接受分数和评分标准（默认为标准 4.0 制），返回对应的绩点值。这个方法被 `CourseService` 在课程创建和更新时调用，实现了绩点的自动计算。

其他 Model 类的共性模式

所有 Model 类（Role、Achievement、Experience、Activity、Goal、Job 等）均遵循相同的模式：

- 默认构造函数 `ClassName() = default;`
- 公共数据成员，带默认初始值
- `toDict()` 实例方法返回 `QJsonObject`
- `static fromDict(const QJsonObject&)` 静态方法返回对象实例
- 多值字段（如 tags、milestones）采用逗号分隔存储 + JSON 数组序列化的双重表示

Goal 类额外提供了 `progress()` 方法，计算目标完成百分比：

```
double progress() const {  
    if (targetValue <= 0) return 0;  
    return round(currentValue / targetValue * 10000.0) / 100.0;  
}
```

该方法在 Goal 的 `toDict()` 中被调用，将进度值一并序列化到 JSON 中。

3.2 DAO 基类与继承体系

DaoBase 基类

DaoBase 定义在 `src/dao/DaoBase.h` 中，是所有 DAO 类的公共基类：

```

class DaoBase {
public:
    DaoBase() : m_db(QSqlDatabase::database()) {
        if (!m_db.isOpen()) {
            Logger::warning("数据库未打开，尝试打开...");
            m_db.open();
        }
    }

    virtual ~DaoBase() = default;

protected:
    QSqlDatabase m_db;

    bool isOpen() const {
        return m_db.isOpen();
    }
};

```

DaoBase 的设计要点：

1. **数据库连接复用**：构造函数通过 `QSqlDatabase::database()` 获取 Qt 的默认数据库连接。由于 SQLite 是文件级数据库，整个应用共享同一个连接实例，DaoBase 的子类不需要关心连接的创建和销毁。
2. **防御性检查**：构造时检查数据库是否已打开，若未打开则尝试打开并输出警告日志。`isOpen()` 方法供子类在执行查询前进行检查。
3. **保护成员**：`m_db` 声明为 `protected`，子类可以直接访问以创建 `QSqlQuery` 对象。

具体 DAO 类

以 `CourseDao` 为例，它继承 `DaoBase` 并提供完整的 CRUD 操作：

```

class CourseDao : public DaoBase {
public:
    // 查询所有课程，按更新时间降序排列
    QList<Course> getAll() const {
        QList<Course> courses;
        if (!isOpen()) return courses;
        QSqlQuery query(m_db);
        query.exec("SELECT * FROM courses ORDER BY updated_at DESC, id DESC");
        while (query.next()) courses.append(mapFromQuery(query));
        return courses;
    }

    // 按 ID 查询单个课程
    Course getById(int id) const {
        Course course;
        if (!isOpen()) return course;
        QSqlQuery query(m_db);
        query.prepare("SELECT * FROM courses WHERE id = :id");
        query.bindValue(":id", id);
        query.exec();
        if (query.next()) course = mapFromQuery(query);
        return course;
    }

    // 插入新课程
    bool create(const Course& course) {
        if (!isOpen()) return false;
        QSqlQuery query(m_db);
        query.prepare(
            "INSERT INTO courses (name, code, credits, semester, ...) "
            "VALUES (:name, :code, :credits, :semester, ...)"
        );
        query.bindValue(":name", course.name);
        // ... 其他字段绑定
        return query.exec();
    }

    // 更新课程
    bool update(const Course& course) {
        if (!isOpen()) return false;
        QSqlQuery query(m_db);
        query.prepare("UPDATE courses SET name = :name, ... WHERE id = :id");
        query.bindValue(":id", course.id);
        // ... 其他字段绑定
    }

```

```

        return query.exec();
    }

    // 删除课程
    bool remove(int id) {
        if (!isOpen()) return false;
        QSqlQuery query(m_db);
        query.prepare("DELETE FROM courses WHERE id = :id");
        query.bindValue(":id", id);
        return query.exec();
    }

private:
    // 将 QSqlQuery 当前行映射为 Course 对象
    static Course mapFromQuery(const QSqlQuery& query) {
        Course c;
        c.id = query.value("id").toInt();
        c.name = query.value("name").toString();
        // ... 其他字段映射
        return c;
    }
};

```

所有 DAO 类共享以下模式：

- 继承 DaoBase 获取 m_db 连接
- 提供 getAll() / getById() / create() / update() / remove() 五个核心 CRUD 方法
- 使用 query.prepare() + query.bindValue() 实现参数化查询，防止 SQL 注入
- 私有 mapFromQuery() 静态方法负责将查询结果行映射为 Model 对象
- 每个方法开头都调用 isOpen() 进行防御性检查

3.3 Service 层

Service 层封装业务逻辑，调用 DAO 层完成数据操作。所有 Service 类均采用静态方法模式，无需实例化即可使用。以 CourseService 为例：

```
class CourseService {
public:
    static QList<Course> getAll() {
        CourseDao dao;
        return dao.getAll();
    }
    static bool create(const Course& course) {
        CourseDao dao;
        return dao.create(course);
    }
    // ...
};
```

AIService 双模式架构

AIService 是系统中最复杂的 Service 类，实现了**远程 AI 模型**和**本地规则**双模式架构。

模式判断机制


```

// 检查远程 AI 服务器是否可用
bool AIService::isAiServerAvailable(bool forceRefresh) {
    const qint64 nowMs = QDateTime::currentMSecsSinceEpoch();
    // 缓存机制: 可用状态缓存 15 秒, 不可用状态 3 秒后重试
    const bool cacheActive = m_aiServerChecked
        && !forceRefresh
        && ((m_aiServerAvailable && (nowMs - m_lastAiServerCheckMs) < kAiAvailableCacheMs)
            || (!m_aiServerAvailable && (nowMs - m_lastAiServerCheckMs) < kAiUnavailableRetryMs));
    if (cacheActive) return m_aiServerAvailable;

    // 发送 HTTP GET 请求到 /health 端点
    QNetworkAccessManager manager;
    QUrl url(m_aiServerUrl + "/health");
    QNetworkRequest request(url);
    request.setTransferTimeout(kAiHealthTimeoutMs); // 2 秒超时

    QEventLoop loop;
    QNetworkReply* reply = manager.get(request);
    QTimer::singleShot(kAiHealthTimeoutMs, &loop, &QEventLoop::quit);
    QObject::connect(reply, &QNetworkReply::finished, &loop, &QEventLoop::quit);
    loop.exec();

    // 检查响应中的 model_loaded 字段
    if (reply->error() == QNetworkReply::NoError) {
        const QJsonObject health = QJsonDocument::fromJson(reply->readAll()).object();
        m_aiServerAvailable = health.value("model_loaded").toBool(false);
    }
    return m_aiServerAvailable;
}

```

分析请求的双模式路由

```

QJsonObject AiService::analyze(const QJsonObject& data) {
    // 优先尝试远程 AI 模型
    if (isAiServerAvailable()) {
        QJsonObject aiResult = analyzeWithAi(data);
        if (!hasAiError(aiResult)) {
            aiResult["aiPowered"] = true;
            return aiResult;
        }
        // 远程调用失败，重置检查状态
        resetAiServerCheck();
    }
    // 回退到本地规则分析
    return analyzeLocal(data);
}

```

本地规则模式

当远程 AI 服务不可用时，analyzeLocal() 根据请求类型分发到不同的本地规则方法：

```

QJsonObject AiService::generateLocalAnalysis(const QString& type) {
    if (type == "course") return buildCourseAdvice();
    if (type == "goal") return buildGoalAdvice();
    if (type == "experience" || type == "career") return buildExperienceAdvice();
    if (type == "resume") return buildResumeAdvice();
    if (type == "achievement") return buildAchievementAdvice();
    if (type == "comprehensive" || type == "general") return buildComprehensiveAdvice();
    return buildDefaultReply();
}

```

以 buildCourseAdvice() 为例，本地规则通过查询数据库统计数据，基于阈值判断生成建议：

```

QJsonObject AiService::buildCourseAdvice() {
    CourseService cs;
    QList<Course> courses = cs.getAll();
    int completed = 0, inProgress = 0;
    double totalGpa = 0;
    int gpaCount = 0;

    for (const Course& c : courses) {
        if (c.status == "Completed") completed++;
        else if (c.status == "In Progress") inProgress++;
        if (c.gradePoint > 0) { totalGpa += c.gradePoint; gpaCount++; }
    }

    double avgGpa = gpaCount > 0 ? totalGpa / gpaCount : 0;
    QStringList tips;
    tips.append(QString("当前共 %1 门课程，已完成 %2 门，进行中 %3 门。")
        .arg(courses.size()).arg(completed).arg(inProgress));

    if (avgGpa < 3.0) {
        tips.append("建议重点关注专业课学习，尝试优化学习方法或参加学习小组。");
    } else if (avgGpa >= 3.5) {
        tips.append("GPA 表现优秀，可考虑挑战更高难度课程或参与科研项目。");
    }
    // ...
}

```

远程 AI 模式

远程模式通过 `callAiServer()` 向本地 Qwen 模型服务器发送 HTTP POST 请求：

```

QJsonObject AiService::callAiServer(const QString& endpoint, const QJsonObject& data) {
    QNetworkAccessManager manager;
    QUrl url(m_aiServerUrl + endpoint); // 默认 http://localhost:8001
    QNetworkRequest request(url);
    request.setHeader(QNetworkRequest::ContentTypeHeader, "application/json");
    request.setTransferTimeout(kAiGenerationTimeoutMs); // 180 秒超时

    QEventLoop loop;
    QNetworkReply* reply = manager.post(request, QJsonDocument(data).toJson());
    // ... 等待响应并解析
}

```

聊天功能的远程模式遵循 OpenAI 兼容的 API 格式：

```

QJsonObject AiService::chatWithAi(const QJsonObject& data) {
    QJsonObject requestData;
    QJsonArray messages;
    messages.append(QJsonObject{{"role", "user"}, {"content", data["message"].toString()}});
    requestData["messages"] = messages;
    requestData["max_tokens"] = 256;
    requestData["temperature"] = 0.3;
    return callAiServer("/v1/chat/completions", requestData);
}

```

3.4 核心控制器类

MainWindow

`MainWindow` 继承 `QMainWindow`，是整个桌面应用的主窗口类。它承担了页面管理、导航切换、系统托盘、全局事件处理等多项职责。

页面管理

`MainWindow` 使用 `QStackedWidget` 管理 13 个功能页面，并通过 `QMap<int, std::function<void()>> m_pageRefreshers` 注册表实现页面刷新的动态分发：

```

// 页面注册表：替代 switch 硬编码
m_overviewPage = new OverviewPage(this);
m_pageRefreshers[m_stack->addWidget(m_overviewPage)] = [this]() { m_overviewPage->refresh(); };

m_coursesPage = new CoursesPage(this);
m_pageRefreshers[m_stack->addWidget(m_coursesPage)] = [this]() { m_coursesPage->refresh(); };

// ... 其他页面注册

void MainWindow::refreshCurrentPage() {
    const int index = m_stack ? m_stack->currentIndex() : 0;
    auto it = m_pageRefreshers.find(index);
    if (it != m_pageRefreshers.end()) {
        it.value()(); // 调用对应的刷新回调
    }
}

```

这种注册表模式的优势在于：添加新页面时只需增加一行注册代码，无需修改 `refreshCurrentPage()` 方法本身，符合开闭原则。

页面切换动画

onNavigationChanged() 方法在页面切换时创建透明度动画：

```
void MainWindow::onNavigationChanged(int row) {
    if (row >= 0 && row < m_stack->count()) {
        m_stack->setCurrentIndex(row);
        m_shellController->onPageChanged(row);

        QWidget* widget = m_stack->widget(row);
        if (widget) {
            // 清除旧效果
            if (widget->graphicsEffect()) {
                widget->graphicsEffect()->setEnabled(false);
                widget->setGraphicsEffect(nullptr);
            }
            // 创建淡入动画
            QGraphicsOpacityEffect* eff = new QGraphicsOpacityEffect(widget);
            widget->setGraphicsEffect(eff);
            QPropertyAnimation* anim = new QPropertyAnimation(eff, "opacity");
            anim->setDuration(120);
            anim->setStartValue(0.0);
            anim->setEndValue(1.0);
            connect(anim, &QPropertyAnimation::finished, widget, [widget]() {
                widget->setGraphicsEffect(nullptr); // 动画结束后移除效果
            });
            anim->start(QAbstractAnimation::DeleteWhenStopped);
        }
        QTimer::singleShot(0, this, &MainWindow::refreshCurrentPage);
    }
}
```

系统托盘

MainWindow 通过 QSystemTrayIcon 实现系统托盘功能：

```

void MainWindow::setupSystemTray() {
    m_trayIcon = new QSystemTrayIcon(this);
    m_trayIcon->setIcon(QApplication::style()->standardIcon(QStyle::SP_ComputerIcon));
    m_trayIcon->setToolTip("学业发展规划系统");

    m_trayMenu = new QMenu(this);
    m_openBrowserAction = m_trayMenu->addAction("打开网页预览");
    m_trayMenu->addSeparator();
    m_quitAction = m_trayMenu->addAction("退出");
    m_trayIcon->setContextMenu(m_trayMenu);

    connect(m_trayIcon, &QSystemTrayIcon::activated, this, &MainWindow::onTrayActivated);
    m_trayIcon->show();
}

```

双击托盘图标可恢复窗口显示，关闭窗口时隐藏到托盘而非直接退出。

QSettings 状态保存

MainWindow 在构造和析构时通过 QSettings 保存和恢复窗口几何状态：

```

// 构造时恢复
QSettings settings;
restoreGeometry(settings.value("geometry").toByteArray());

// 析构时保存
QSettings settings;
settings.setValue("geometry", saveGeometry());

```

BackendRuntimeController

BackendRuntimeController 管理嵌入式 HTTP 服务器的生命周期。它持有一个 HttpServerThread 指针，通过信号槽与 UI 组件绑定：

```

class BackendRuntimeController : public QObject {
    Q_OBJECT
public:
    void bindWidgets(QLabel* statusLabel, QProgressBar* progressBar, QSystemTrayIcon* trayIcon);
    void startBackendServer();
    void stopBackendServer();
    void openBrowser();
    void checkFrontendExists();
    void insertSampleDataIfNeeded();

signals:
    void serverStarted();
    void serverError(const QString& error);
    void backendReady();
};

```

在 MainWindow 构造时，BackendRuntimeController 被绑定到状态标签、进度条和托盘图标：

```

m_backendController->bindWidgets(m_statusLabel, m_progressBar, m_trayIcon);
m_backendController->checkFrontendExists();
m_backendController->startBackendServer();

```

DataRefreshCoordinator

DataRefreshCoordinator 是数据刷新的核心协调器，基于 DataDomain 枚举实现按域刷新：

```

enum class DataDomain {
    Courses, Goals, Roles, Achievements, Experiences,
    Activities, Jobs, Analysis, Resume, Timeline, All
};

```

DataRefreshCoordinator 持有所有 12 个页面的指针，通过 bindPages() 绑定，connectSignals() 连接信号，refreshByDomain() 按域分发刷新：

```

class DataRefreshCoordinator : public QObject {
    Q_OBJECT
public:
    void bindPages(OverviewPage*, CoursesPage*, RolesPage*, /* ... 所有页面 */);
    void connectSignals();
    void refreshByDomain(DataDomain domain);
    void refreshAll();
};

```

当任意页面执行数据修改操作后，发出 `dataChanged(DataDomain)` 信号，`DataRefreshCoordinator` 接收到信号后调用对应域的页面 `refresh()` 方法，实现数据变更的自动传播。

AiContextMediator

`AiContextMediator` 实现了事件过滤器模式，拦截全局的文本选中事件：

```

class AiContextMediator : public QObject {
    Q_OBJECT
public:
    void bindRootWidget(QWidget* root);
    void attachPanel(AiPanelWidget* panel);
    bool eventFilter(QObject* watched, QEvent* event) override;
    void pushSelectionToPanel(const QString& text);
};

```

它通过 `eventFilter()` 监听全局鼠标事件，当检测到用户选中文本时，将选中内容推送到 AI 面板，用户可以直接基于选中文本发起 AI 分析。

CrudPageController

`CrudPageController` 使用 `std::function` 回调实现策略模式，将对话框创建和服务调用的逻辑通过依赖注入解耦：


```

class CrudPageController : public QObject {
    Q_OBJECT
public:
    using CreateDialogFn = std::function<QJsonObject(QWidget*)>;
    using EditDialogFn = std::function<QJsonObject(QWidget*, const QJsonObject*)>;
    using ServiceCreateFn = std::function<void(const QJsonObject*)>;
    using ServiceUpdateFn = std::function<void(int, const QJsonObject*)>;
    using ServiceRemoveFn = std::function<void(int)>;

    void setCreateDialog(CreateDialogFn fn);
    void setServiceCreate(ServiceCreateFn fn);
    // ...

    void executeAdd(QWidget* parent);
    void executeEdit(QWidget* parent, int id);
    void executeRemove(QWidget* parent, int id);

signals:
    void dataChanged(int domain);
};

```

使用时，各页面通过 `set*` 方法注入具体的对话框和服务实现：

```

// 以 CoursesPage 为例
m_crudController->setCreateDialog([](QWidget* parent) -> QJsonObject {
    CourseEditorDialog dlg(parent);
    if (dlg.exec() == QDialog::Accepted) return dlg.getResult();
    return QJsonObject();
});
m_crudController->setServiceCreate([](const QJsonObject& data) {
    CourseService::create(Course::fromDict(data));
});

```

3.5 HttpServer 与 API 路由注册

`HttpServer` 类定义在 `src/server/HttpServer.h` 中，基于 `QHttpServer` + `QTcpServer` 组合实现 HTTP 服务器：

```

class HttpServer {
public:
    bool start(quint16 port = 8080) {
        registerRoutes();
        m_tcpServer.listen(QHostAddress::Any, port);
        if (!m_server.bind(&m_tcpServer)) {
            Logger::error(QString("服务器启动失败，端口 %1 可能已被占用").arg(port));
            return false;
        }
        return true;
    }

private:
    void registerRoutes() {
        // 全局 CORS 头注入
        m_server.addAfterRequestHandler(&m_server, [](const QHttpRequest&, QHttpResponse& resp) {
            JsonUtils::applyCommonHeaders(resp);
        });

        // 静态文件路由
        m_server.route("/", QHttpRequest::Method::Get,
            [this]() { return staticFileResponse("index.html"); });

        // RESTful API 路由（以课程为例）
        m_server.route("/api/courses", QHttpRequest::Method::Get,
            [](const QHttpRequest& req) { return CourseApi::getAll(req); });
        m_server.route("/api/courses", QHttpRequest::Method::Post,
            [](const QHttpRequest& req) { return CourseApi::create(req); });
        m_server.route("/api/courses/<arg>", QHttpRequest::Method::Get,
            [](int id, const QHttpRequest&) { return CourseApi::getId(id); });
        m_server.route("/api/courses/<arg>", QHttpRequest::Method::Put,
            [](int id, const QHttpRequest& req) { return CourseApi::update(id, req); });
        m_server.route("/api/courses/<arg>", QHttpRequest::Method::Delete,
            [](int id, const QHttpRequest&) { return CourseApi::remove(id); });
        // ... 其他路由
    }

    QHttpServer m_server;
    QTcpServer m_tcpServer;
};

```

静态文件服务

staticFileResponse() 方法在多个候选路径中查找 frontend_dist 目录，找到后通过 QHttpServerResponse::fromFile() 返回静态文件：

```
QHttpServerResponse staticFileResponse(const QString& relativePath) const {
    const QStringList candidates = {
        QDir(QCoreApplication::applicationDirPath()).absoluteFilePath("frontend_dist"),
        QDir(QCoreApplication::applicationDirPath()).absoluteFilePath("../frontend_dist"),
        QDir::current().absoluteFilePath("frontend_dist")
    };
    for (const QString& candidate : candidates) {
        if (QFileInfo::exists(QDir(candidate).absoluteFilePath("index.html"))) {
            return QHttpServerResponse::fromFile(QDir(candidate).absoluteFilePath(relativePath));
        }
    }
    return JsonUtils::errorResponse("前端静态资源不存在", 404);
}
```

API 路由总览

系统注册了以下 RESTful API 路由：

路径	方法	处理器
/api/courses	GET/POST	CourseApi::getAll / create
/api/courses/<id>	GET/PUT/DELETE	CourseApi::getById / update / remove
/api/courses/statistics	GET	CourseApi::getStatistics
/api/roles	GET/POST	RoleApi::getAll / create
/api/roles/<id>	GET/PUT/DELETE	RoleApi::getById / update / remove
/api/achievements	GET/POST	AchievementApi::getAll / create
/api/achievements/<id>	GET/PUT/DELETE	AchievementApi::getById / update / remove
/api/experiences	GET/POST	ExperienceApi::getAll / create
/api/experiences/<id>	GET/PUT/DELETE	ExperienceApi::getById / update / remove
/api/activities	GET/POST	ActivityApi::getAll / create
/api/activities/<id>	GET/PUT/DELETE	ActivityApi::getById / update / remove

路径	方法	处理器
/api/goals	GET/POST	GoalApi::getAll / create
/api/goals/<id>	GET/PUT/DELETE	GoalApi::getById / update / remove
/api/jobs	GET/POST	JobApi::getAll / create
/api/jobs/<id>	GET/PUT/DELETE	JobApi::getById / update / remove
/api/jobs/import	POST	JobApi::importJobs
/api/dashboard/overview	GET	DashboardApi::getOverview
/api/dashboard/gpa-trend	GET	DashboardApi::getGpaTrend
/api/dashboard/recommendations	GET	DashboardApi::getRecommendations
/api/analytics/semester-comparison	GET	AnalyticsApi::getSemesterComparison
/api/analytics/peer-comparison	GET	AnalyticsApi::getPeerComparison
/api/analytics/peers	GET/POST	AnalyticsApi::getPeers / createPeer
/api/analytics/peers/<id>	PUT/DELETE	AnalyticsApi::updatePeer / deletePeer
/api/timeline	GET	TimelineApi::getAll
/api/imports/<entity>	POST	ImportApi::importEntity
/api/resume/generate	POST	ResumeApi::generate
/api/resume/export/json	POST	ResumeApi::exportJson
/api/resume/export/html	POST	ResumeApi::exportHtml
/api/ai/analyze	POST	AiApi::analyze
/api/ai/status	GET	AiApi::checkStatus
/api/ai/chat	POST	AiApi::chat
/api/auth/register	POST	AuthApi::registerUser
/api/auth/login	POST	AuthApi::login
/api/auth/me	GET/PUT	AuthApi::getMe / updateMe
/api/auth/change-password	POST	AuthApi::changePassword
/	GET	静态文件 index.html

路径	方法	处理器
/assets/<file>	GET	静态资源文件

四、关键组件使用细节

4.1 QTableWidgetItem 使用

QTableWidgetItem 是系统中最核心的数据展示组件，广泛应用于课程管理、成就管理、经历管理等页面。以 CoursesPage 为例，其典型使用模式如下：

```
QTableWidgetItem* table = new QTableWidgetItem(this);
table->setColumnCount(7);
table->setHorizontalHeaderLabels({"课程名称", "代码", "学分", "学期", "成绩", "绩点", "状态"});
table->setSelectionBehavior(QAbstractItemView::SelectRows);
table->setSelectionMode(QAbstractItemView::SingleSelection);
table->setEditTriggers(QAbstractItemView::NoEditTriggers);
table->horizontalHeader()->setStretchLastSection(true);
table->setSortingEnabled(true);
```

数据填充时，遍历 Service 层返回的 Model 列表，逐行设置单元格：

```
QList<Course> courses = CourseService::getAll();
table->setRowCount(courses.size());
for (int i = 0; i < courses.size(); ++i) {
    table->setItem(i, 0, new QTableWidgetItem(courses[i].name));
    table->setItem(i, 1, new QTableWidgetItem(courses[i].code));
    table->setItem(i, 2, new QTableWidgetItem(QString::number(courses[i].credits)));
    // ...
}
```

通过 setSortingEnabled(true) 启用列排序，用户点击表头即可按该列排序。右键菜单通过重写 contextMenuEvent() 实现编辑和删除操作。

4.2 QHttpServer 路由注册

QHttpServer 的路由注册使用 route() 模板方法，支持路径参数捕获和 HTTP 方法限定：

```

// 无参数路由
m_server.route("/api/courses", QHttpRequest::Method::Get,
    [] (const QHttpRequest& req) { return CourseApi::getAll(req); });

// 带路径参数路由 (<arg> 自动转换为 int)
m_server.route("/api/courses/<arg>", QHttpRequest::Method::Get,
    [] (int id, const QHttpRequest&) { return CourseApi::getById(id); });

// 多参数路由
m_server.route("/api/jobs/<arg>/requirements/<arg>/toggle", QHttpRequest::Method::Post,
    [] (int jobId, int reqIndex, const QHttpRequest&) {
        return JobApi::toggleRequirement(jobId, reqIndex);
    });

```

全局请求后处理器通过 `addAfterRequestHandler()` 注入 CORS 头：

```

m_server.addAfterRequestHandler(&m_server, [] (const QHttpRequest&, QHttpResponse& resp
    JsonUtils::applyCommonHeaders(resp);
});

```

4.3 QSqlQuery 参数绑定

系统中所有 SQL 查询均使用参数化语句，通过 `prepare()` 预编译 SQL 模板，`bindValue()` 绑定参数值：

```

QSqlQuery query(m_db);
query.prepare(
    "INSERT INTO courses (name, code, credits, semester, category, score, grade_point, status, "
    "teacher, location, description, tags) "
    "VALUES (:name, :code, :credits, :semester, :category, :score, :grade_point, :status, "
    " :teacher, :location, :description, :tags)"
);
query.bindValue(":name", course.name);
query.bindValue(":code", course.code);
query.bindValue(":credits", course.credits);
query.bindValue(":score", course.score > 0 ? QVariant(course.score) : QVariant());
query.bindValue(":grade_point", course.gradePoint > 0 ? QVariant(course.gradePoint) : QVariant());
query.exec();

```

对于可空字段（如 `score`、`grade_point`），使用三元表达式在有值时绑定实际值、无值时绑定 `QVariant()`（即 `NULL`）。

参数化查询的核心优势在于防止 SQL 注入。由于用户输入通过 `bindValue()` 传递而非字符串拼接，SQLite 引擎会自动对参数值进行转义处理。

4.4 QSettings 持久化

系统使用 QSettings 实现应用配置的持久化存储。QSettings 在不同平台上使用不同的后端（Windows 使用注册表，macOS 使用 plist，Linux 使用 INI 文件），Qt 自动处理平台差异。

MainWindow 中使用 QSettings 保存窗口几何状态：

```
// 构造时恢复
QSettings settings;
restoreGeometry(settings.value("geometry").toByteArray());

// 析构时保存
QSettings settings;
settings.setValue("geometry", saveGeometry());
```

`saveGeometry()` 返回 `QByteArray`，包含窗口位置和大小信息；`restoreGeometry()` 在下次启动时恢复到上次关闭时的窗口状态。

4.5 QCryptographicHash 密码哈希

AuthService 使用 QCryptographicHash 实现密码的安全存储：

```
static QString hashPassword(const QString& password) {
    QByteArray hash = QCryptographicHash::hash(
        (password + "pdp_salt_2024").toUtf8(),
        QCryptographicHash::Sha256
    );
    return QString(hash.toHex());
}

static bool verifyPassword(const QString& password, const QString& hash) {
    return hashPassword(password) == hash;
}
```

密码哈希流程：

1. 将用户输入的密码与固定盐值 "pdp_salt_2024" 拼接
2. 转换为 UTF-8 字节数组
3. 使用 SHA-256 算法计算哈希值

4. 将哈希值转换为十六进制字符串存储到数据库的 password_hash 字段

验证时重新计算哈希并与存储值比较。Token 生成同样基于 SHA-256，结合用户 ID、用户名、角色和过期时间生成：

```
static QString generateToken(int userId, const QString& username, const QString& role) {
    QString data = QString("%1:%2:%3:%4")
        .arg(userId).arg(username).arg(role)
        .arg(QDateTime::currentSecsSinceEpoch() + 72 * 3600); // 72 小时有效期
    QByteArray hash = QCryptographicHash::hash(data.toUtf8(), QCryptographicHash::Sha256);
    return QString("%1.%2.%3")
        .arg(QString(QByteArray::number(userId).toBase64()))
        .arg(QString(QCryptographicHash::hash(username.toUtf8(), QCryptographicHash::Md5).toHex()))
        .arg(QString(hash.toHex()).left(24));
}
```

4.6 std::sort 排序

系统在数据分析和展示场景中使用 std::sort 配合 lambda 比较器实现自定义排序：

```
// 按 GPA 降序排序课程
std::sort(courses.begin(), courses.end(), [](const Course& a, const Course& b) {
    return a.gradePoint > b.gradePoint;
});

// 按优先级排序目标 (High > Medium > Low)
std::sort(goals.begin(), goals.end(), [](const Goal& a, const Goal& b) {
    static QMap<QString, int> priorityMap = {"High", 3}, {"Medium", 2}, {"Low", 1};
    return priorityMap.value(a.priority, 0) > priorityMap.value(b.priority, 0);
});
```

lambda 比较器返回 true 表示第一个参数应排在第二个参数之前。

4.7 std::function 回调解耦

std::function 在系统中有两个典型应用场景：

CrudPageController 的策略注入

通过 std::function 定义函数签名类型别名，将对话框创建和 CRUD 操作的具体实现通过 setter 方法注入：


```

using CreateDialogFn = std::function<QJsonObject(QWidget*)>;
using EditDialogFn = std::function<QJsonObject(QWidget*, const QJsonObject*)>;
using ServiceCreateFn = std::function<void(const QJsonObject*)>;
using ServiceUpdateFn = std::function<void(int, const QJsonObject*)>;
using ServiceRemoveFn = std::function<void(int)>;

```

每个页面在初始化时通过 lambda 表达式注入具体的实现，使得 CrudPageController 可以复用于所有 CRUD 页面而无需了解具体的业务细节。

MainWindow 的页面刷新注册表

```

QMap<int, std::function<void()>> m_pageRefreshers;

// 注册
m_pageRefreshers[m_stack->addWidget(m_coursesPage)] = [this]() { m_coursesPage->refresh(); };

// 调用
auto it = m_pageRefreshers.find(index);
if (it != m_pageRefreshers.end()) it.value()();

```

4.8 QSystemTrayIcon 系统托盘

MainWindow 通过 QSystemTrayIcon 实现系统托盘功能：

```

void MainWindow::setupSystemTray() {
    if (!QSystemTrayIcon::isSystemTrayAvailable()) {
        Logger::warning("系统托盘不可用");
        return;
    }

    m_trayIcon = new QSystemTrayIcon(this);
    m_trayIcon->setIcon(QApplication::style()->standardIcon(QStyle::SP_ComputerIcon));
    m_trayIcon->setToolTip("学业发展规划系统");

    // 托盘右键菜单
    m_trayMenu = new QMenu(this);
    m_openBrowserAction = m_trayMenu->addAction("打开网页预览");
    m_trayMenu->addSeparator();
    m_quitAction = m_trayMenu->addAction("退出");
    m_trayIcon->setContextMenu(m_trayMenu);

    // 双击恢复窗口
    connect(m_trayIcon, &QSystemTrayIcon::activated, this, &MainWindow::onTrayActivated);
    m_trayIcon->show();
}

void MainWindow::onTrayActivated(QSystemTrayIcon::ActivationReason reason) {
    if (reason == QSystemTrayIcon::DoubleClick) {
        show();
        raise();
        activateWindow();
    }
}

```

关闭窗口时隐藏到托盘而非退出：

```

void MainWindow::closeEvent(QCloseEvent* event) {
    if (m_trayIcon) m_trayIcon->hide();
    event->accept();
    qApp->quit();
}

```

4.9 QGraphicsOpacityEffect 动画

页面切换时的淡入效果通过 QGraphicsOpacityEffect + QPropertyAnimation 实现：

```
QGraphicsOpacityEffect* eff = new QGraphicsOpacityEffect(widget);
widget->setGraphicsEffect(eff);

QPropertyAnimation* anim = new QPropertyAnimation(eff, "opacity");
anim->setDuration(120);           // 120 毫秒
anim->setStartValue(0.0);         // 从完全透明
anim->setEndValue(1.0);           // 到完全不透明

// 动画结束后移除效果，避免影响后续渲染
connect(anim, &QPropertyAnimation::finished, widget, [widget]() {
    widget->setGraphicsEffect(nullptr);
});

anim->start(QAbstractAnimation::DeleteWhenStopped);
```

关键设计决策：动画结束后通过 `setGraphicsEffect(nullptr)` 移除效果对象。这是因为 `QGraphicsOpacityEffect` 会强制将 `widget` 渲染到离屏缓冲区再进行合成，长期保留会影响渲染性能。

五、面向对象设计要点

5.1 分层架构

系统采用六层分层架构，各层职责明确：

层级	职责	典型类
Model 层	定义数据结构，提供序列化方法	Course, Goal, Achievement
DAO 层	封装数据库访问，提供 CRUD 操作	CourseDao, GoalDao
Service 层	实现业务逻辑，编排 DAO 调用	CourseService, AiService
API 层	解析 HTTP 请求，构建 HTTP 响应	CourseApi, AiApi
Controller 层	协调 UI 与业务逻辑的流程	DataRefreshCoordinator
UI 层	用户交互展示	CoursesPage, CourseEditorDialog

层间依赖方向严格向下：UI 层依赖 Controller 层和 Service 层，Controller 层依赖 Service 层，Service 层依赖 DAO 层，DAO 层依赖 Model 层。API 层依赖 Service 层和 Model 层。这种单向依赖确保了各层的独立性和可测试性。

5.2 数据模型序列化模式

所有 Model 类遵循统一的 `toDict()` / `fromDict()` 双方法序列化模式：

- `toDict()` 是实例方法，将当前对象转换为 `JsonObject`
- `fromDict()` 是静态方法，从 `JsonObject` 创建新对象

`JsonObject` 作为中间格式的优势：

1. **跨层传递**：API 层可以直接将 Model 转为 JSON 返回给客户端，Service 层可以从 JSON 构造 Model
2. **类型安全**：编译期类型检查确保字段访问正确
3. **灵活性**：JSON 格式天然适配 RESTful API 和前端数据绑定

多值字段（tags、milestones）的双重表示策略（数据库逗号分隔、JSON 数组）兼顾了存储简洁性和前端易用性。

5.3 DAO 基类继承模式

DaoBase 基类通过以下设计实现代码复用和连接管理：

1. **构造函数自动获取连接**：子类无需关心数据库连接的创建
2. **保护成员 `m_db`**：子类直接访问，避免虚函数调用开销
3. **`isOpen()` 防御性检查**：每个 CRUD 方法开头调用，确保数据库可用
4. **私有 `mapFromQuery()` 方法**：将 `ResultSet` 映射逻辑封装在 DAO 内部，对外部不可见

子 DAO 类的 CRUD 方法全部为非虚方法，这意味着 DAO 层不使用多态，而是通过组合（Service 持有具体 DAO 实例）而非继承来扩展行为。

5.4 信号槽驱动的数据刷新

数据刷新采用观察者模式，以 `DataDomain` 枚举为核心：

数据变更 → 发出 `dataChanged(DataDomain)` 信号
→ `DataRefreshCoordinator` 接收信号
→ `refreshByDomain(domain)` 分发到对应页面
→ 页面.`refresh()` 重新加载数据

`DataDomain` 枚举定义了 11 个数据域，覆盖系统的所有功能模块。这种基于枚举的域划分使得数据刷新可以精确到具体模块，避免不必要的全量刷新。

`DataRefreshCoordinator` 的 `connectSignals()` 方法将各页面的 `dataChanged` 信号连接到自身的 `refreshByDomain()` 槽，形成星型拓扑的信号传播网络。

5.5 控制器模式

系统中使用了多种控制器模式：

BackendRuntimeController：管理服务器生命周期，将服务器的启动/停止/状态变化封装为信号，UI 层通过信号槽响应状态变化，实现了服务器管理与 UI 展示的解耦。

CrudPageController：通过 `std::function` 回调实现策略模式，将"创建什么对话框"和"调用什么服务"的决策延迟到页面初始化时注入。这使得同一控制器可以复用于所有 CRUD 页面。

AppShellController：管理应用外壳的视觉状态（顶栏标题、内容宽度、间距），在页面切换时自动更新，使各页面无需关心外壳布局的细节。

5.6 双模式 AI 架构

AIService 的双模式架构体现了**策略模式**和**优雅降级**的设计思想：

1. **优先远程**：`analyze()` 方法首先检查远程 AI 服务器是否可用，可用则调用 `analyzeWithAi()`
2. **失败回退**：远程调用返回错误时，调用 `resetAiServerCheck()` 重置健康检查缓存，然后回退到 `analyzeLocal()`
3. **本地规则**：`analyzeLocal()` 通过查询数据库统计数据，基于阈值规则生成建议
4. **健康检查缓存**：可用状态缓存 15 秒，不可用状态 3 秒后重试，避免频繁的健康检查请求

这种设计确保了系统在任何环境下都能提供 AI 分析功能——有大模型时获得高质量分析，无大模型时获得基于规则的基本建议。

5.7 事件过滤器模式

AiContextMediator 通过 `eventFilter()` 实现全局事件拦截：

```
bool AiContextMediator::eventFilter(QObject* watched, QEvent* event) {  
    // 拦截鼠标事件，检测文本选中  
    // 将选中文本推送到 AI 面板  
}
```

通过在根 Widget 上安装事件过滤器，AiContextMediator 可以监听整个窗口的鼠标事件，而不需要在每个可选中文本的组件上单独绑定事件处理器。这种集中式的事件处理减少了组件间的耦合。

5.8 BasePage 继承体系

所有功能页面继承自 BasePage 基类：

```
class BasePage : public QWidget {
    Q_OBJECT
public:
    explicit BasePage(QWidget* parent = nullptr);
    ~BasePage() override;
    virtual void refresh() = 0; // 纯虚函数，子类必须实现

protected:
    QFrame* createMetricCard(const QString& labelText, QLabel** valueLabel,
                            const QString& helperText = QString());
};
```

refresh() 纯虚函数定义了页面的数据刷新接口，DataRefreshCoordinator 通过调用此方法触发页面数据更新。createMetricCard() 提供了创建指标卡片的统一工具方法，保证了各页面视觉风格的一致性。

六、附录：项目文件结构

6.1 整体目录结构

```
Qt_Cpp_Final_Submission/
├── 01_SourceCode/                                # 主源码目录
│   ├── CMakeLists.txt                            # CMake 构建配置
│   ├── VERSION                                    # 版本号文件（0.3.0）
│   ├── README.md
│   ├── CHANGELOG.md
│   ├── ai_server.py                              # Python AI 服务器（Qwen 模型桥接）
│   ├── qwen_server.py                            # Python Qwen 模型服务器
│   ├── seed_db.py                                # 数据库种子脚本
│   ├── requirements.txt                           # Python 依赖
│   ├── build_desktop.bat                          # 桌面端构建脚本
│   ├── build_frontend.bat                         # 前端构建脚本
│   ├── start.bat / start_desktop.bat             # 启动脚本
│   ├── pdp.db                                    # SQLite 数据库文件
│   ├── docs/                                     # 项目文档
│   ├── frontend_dist/                            # Vue3 前端构建产物
│   ├── resources/                                # Qt 资源文件
│   ├── src/                                       # C++ 源码
│   └── tests/                                    # 测试文件
├── 02_Executable/                                # 预构建可执行文件
├── 03_Reports/                                    # 设计报告
├── 04_Presentation/                              # 答辩材料
└── 05_RunGuide/                                  # 运行指南
```

6.2 各模块文件组织

src/model/ — 数据模型层

Course.h / Course.cpp	# 课程模型（唯一有 .cpp 的 Model 类，包含 calculateGradePoint 实现）
Role.h	# 角色任职模型
Achievement.h	# 成就荣誉模型
Experience.h	# 实践经历模型
Activity.h	# 课外活动模型
Goal.h	# 目标规划模型
Job.h	# 目标岗位模型
JobRequirement.h	# 岗位技能需求模型
PeerBenchmark.h	# 同学对照模型
User.h	# 用户账户模型
ImportResult.h	# 导入结果模型
ImportErrorItem.h	# 导入错误条目模型

src/dao/ — 数据访问层

DaoBase.h / DaoBase.cpp	# DAO 基类
CourseDao.h	# 课程 DAO
RoleDao.h	# 角色 DAO
AchievementDao.h	# 成就 DAO
ExperienceDao.h	# 经历 DAO
ActivityDao.h	# 活动 DAO
GoalDao.h	# 目标 DAO
JobDao.h	# 岗位 DAO
PeerBenchmarkDao.h	# 同学对照 DAO
UserDao.h	# 用户 DAO

src/service/ — 业务逻辑层

AiService.h / AiService.cpp	# AI 服务（唯一有 .cpp 的 Service 类）
CourseService.h	# 课程服务
RoleService.h	# 角色服务
AchievementService.h	# 成就服务
ExperienceService.h	# 经历服务
ActivityService.h	# 活动服务
GoalService.h	# 目标服务
JobService.h	# 岗位服务
DashboardService.h	# 仪表盘服务
ImportService.h	# 导入服务
AuthService.h	# 认证服务
AnalyticsService.h	# 分析服务（包含 PeerBenchmarkService）
ResumeService.h	# 简历服务

src/api/ — API 层

CourseApi.h	# 课程 API
RoleApi.h	# 角色 API
AchievementApi.h	# 成就 API
ExperienceApi.h	# 经历 API
ActivityApi.h	# 活动 API
GoalApi.h	# 目标 API
JobApi.h	# 岗位 API
DashboardApi.h	# 仪表盘 API
AnalyticsApi.h	# 分析 API
TimelineApi.h	# 时间线 API
ImportApi.h	# 导入 API
ResumeApi.h	# 简历 API
AiApi.h	# AI API
AuthApi.h	# 认证 API

src/server/ — HTTP 服务器层

HttpServer.h	# HTTP 服务器（QHttpServer + QTcpServer 封装）
--------------	---

src/client/core/ — 控制器层

DataDomain.h	# 数据域枚举定义
AppShellController.h	# 应用外壳控制器
DataRefreshCoordinator.h	# 数据刷新协调器
BackendRuntimeController.h	# 后端运行时控制器
AiContextMediator.h	# AI 上下文中介器
CrudPageController.h	# CRUD 页面通用控制器

src/client/pages/ — 页面层

BasePage.h / BasePage.cpp	# 页面基类
OverviewPage.h / OverviewPage.cpp	# 总览页面
CoursesPage.h / CoursesPage.cpp	# 课程管理页面
RolesPage.h / RolesPage.cpp	# 角色任职页面
AchievementsPage.h / .cpp	# 成就荣誉页面
ExperiencesPage.h / .cpp	# 实践经历页面
ActivitiesPage.h / .cpp	# 课外活动页面
GoalsPage.h / GoalsPage.cpp	# 目标规划页面
JobsPage.h / JobsPage.cpp	# 岗位追踪页面
AnalysisPage.h / AnalysisPage.cpp	# 数据分析页面
TimelinePage.h / TimelinePage.cpp	# 时间线页面
ResumePage.h / ResumePage.cpp	# 简历生成页面
ImportsPage.h / ImportsPage.cpp	# 数据导入页面

src/client/dialogs/ — 对话框层

ProfileEditorDialog.h / .cpp	# 个人信息编辑
CourseEditorDialog.h / .cpp	# 课程编辑
RoleEditorDialog.h / .cpp	# 角色编辑
AchievementEditorDialog.h / .cpp	# 成就编辑
ExperienceEditorDialog.h / .cpp	# 经历编辑
ActivityEditorDialog.h / .cpp	# 活动编辑
GoalEditorDialog.h / .cpp	# 目标编辑
JobEditorDialog.h / .cpp	# 岗位编辑
PeerEditorDialog.h / .cpp	# 同学数据编辑

src/client/widgets/ — 组件层

SidebarWidget.h / .cpp	# 侧边栏
NavigationListWidget.h / .cpp	# 导航列表
AIPanelWidget.h / .cpp	# AI 面板
AIConversationWidget.h / .cpp	# AI 对话
AiStatusBar.h / .cpp	# AI 状态栏
ToastNotification.h / .cpp	# 消息提示
TimeInfoCard.h / .cpp	# 时间信息卡片
StudentInfoCard.h / .cpp	# 学生信息卡片
MetricGridWidget.h / .cpp	# 指标网格
SuggestionListWidget.h / .cpp	# 建议列表
CrudPageShell.h / .cpp	# CRUD 页面外壳
SemesterAnalysisWidget.h / .cpp	# 学期分析
PeerBenchmarkWidget.h / .cpp	# 同学对照
ResumePreviewWidget.h / .cpp	# 简历预览
ResumePreviewDialog.h / .cpp	# 简历预览对话框
ResumeEditorPanel.h / .cpp	# 简历编辑面板
ResumeCandidatePanel.h / .cpp	# 简历候选面板

src/client/utils/ — UI 工具

UiHelpers.h / UiHelpers.cpp	# UI 辅助工具函数
-----------------------------	-------------

src/util/ — 通用工具

Logger.h	# 日志工具（info/warning/error/debug 四级）
JsonUtils.h	# JSON 响应构建工具（统一的 successResponse/errorResponse 格式）

src/config/ — 配置

Version.h.in	# CMake 版本号模板，构建时注入 PDP_VERSION 宏
--------------	-----------------------------------

resources/ — Qt 资源

schema.sql	# 数据库建表脚本（10 张表）
resources.qrc	# Qt 资源文件声明
icons/	# 图标资源